

• 3 •

Introductory Digital Control

A Perspective on Introductory Digital Control

The continuous controllers you have studied so far are built using analog electronics such as resistors, capacitors, and operational amplifiers. However, most control systems today use digital computers (usually microprocessors or microcontrollers) with the necessary input/output hardware to implement the controllers. The intent of this chapter is to show the very basic ideas of designing control laws that will be implemented in a digital computer. Unlike analog electronics, digital computers cannot integrate. Therefore, in order to solve a differential equation in a computer, the equation must be approximated by reducing it to an algebraic equation involving sums and products only. These approximation techniques are often referred to as **numerical integration**. This chapter shows a simple way to make these approximations as an introduction to digital control. Later chapters expand on various improvements to these approximations, show how to analyze them, and show that digital compensation may also be carried out directly without resorting to these approximations. In the final analysis, we will see that direct digital design provides the designer with the most accurate method and the most flexibility in selection of the sample rate.

From the material in this chapter, you should be able to design and implement a digital control system. The system would be expected to give adequate performance if the sample rate is at least 30 times faster than the bandwidth of the system.

Chapter Overview

In Section 3.1, you will learn how to approximate a continuous $D(s)$ with a set of difference equations, a design method sometimes referred to as **emulation**. Section 3.1 is sufficient to enable you to approximate a continuous feedback controller in a digital control system. Section 3.2 shows the basic effect of

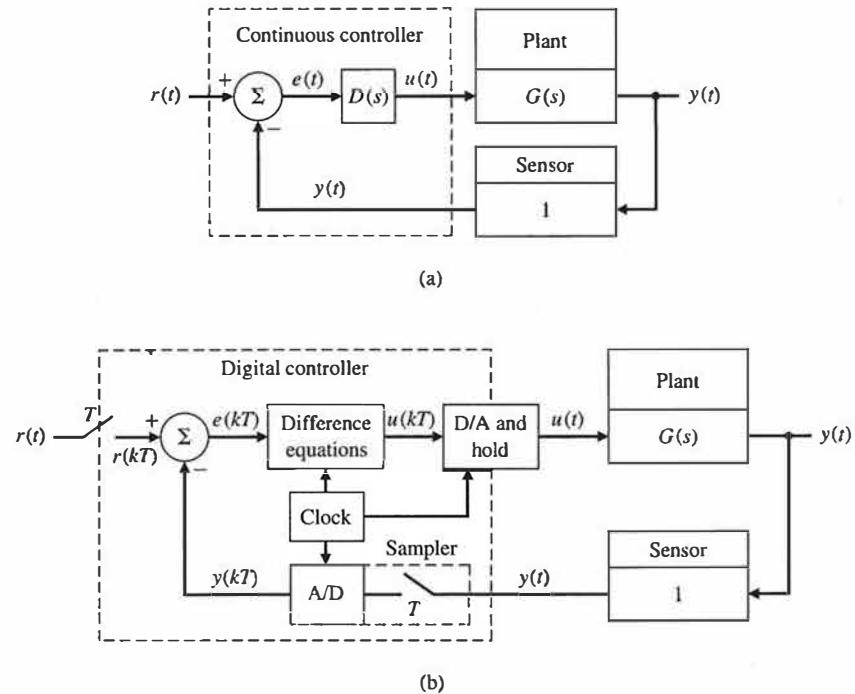
sampling on the performance of the system and a simple way to analyze that effect. Section 3.3 shows how to convert a continuous PID control law to the digital form.

3.1 Digitization

Figure 3.1(a) shows the topology of the typical continuous system. The computation of the error signal, e , and the dynamic compensation, $D(s)$, can all be accomplished in a digital computer as shown in Fig. 3.1(b). The fundamental differences between the two implementations are that the digital system operates on *samples* of the sensed plant output rather than on the continuous signal and that the dynamics represented by $D(s)$ are implemented by algebraic recursive equations called **difference equations**.

We consider first the action of the analog-to-digital (A/D) converter on a signal. This device acts on a physical variable, most commonly an electrical voltage, and converts it into a binary number that usually consists of 10 or 12 bits. A binary number with 10 bits can take on $2^{10} = 1024$ values; therefore, an A/D converter with 10 bits has a resolution of 0.1%. The conversion from the analog signal $y(t)$ occurs repetitively at instants of time that are T seconds

Figure 3.1
Basic control-system
block diagrams:
(a) continuous system,
(b) with a digital
computer



sample period
sample rate

apart. T is called the **sample period** and $1/T$ is the **sample rate** in cycles per second or Hz (also sometimes given in radians/second or $2\pi/T$). The sampled signal is $y(kT)$ where k can take on any integer value. It is often written simply as $y(k)$. We call this type of variable a **discrete signal** to distinguish it from a continuous variable like $y(t)$, which changes continuously in time. We make the assumption here that the sample period is fixed; however, it may vary depending on the implementation as discussed in Section 1.1.

ZOH

There also may be a sampler and A/D converter for the input command, $r(t)$, producing the discrete $r(kT)$ from which the sensed $y(kT)$ would be subtracted to arrive at the discrete error signal, $e(kT)$. The differential equation of the continuous compensation is approximated by a difference equation which is the discrete approximation to the differential equation and can be made to duplicate the dynamic behavior of a $D(s)$ if the sample period is short enough. The result of the difference equation is a discrete $u(kT)$ at each sample instant. This signal is converted to a continuous $u(t)$ by the D/A and hold. The D/A converts the binary number to an analog voltage, and a **zero-order hold (ZOH)** maintains that same voltage throughout the sample period. The resulting $u(t)$ is then applied to the actuator in precisely the same manner as the continuous implementation.

Euler's method

One particularly simple way to make a digital computer approximate the real time solution of differential equations is to use Euler's method. It follows from the definition of a derivative that

$$\dot{x} = \lim_{\delta t \rightarrow 0} \frac{\delta x}{\delta t} \quad (3.1)$$

where δx is the change in x over a time interval δt . Even if δt is not quite equal to zero, this relationship will be approximately true, and

$$\dot{x}(k) \cong \frac{x(k+1) - x(k)}{T} \quad (3.2)$$

where

$T = t_{k+1} - t_k$ (the sample interval in seconds),

$t_k = kT$ (for a constant sample interval),

k is an integer,

$x(k)$ is the value of x at t_k , and

$x(k+1)$ is the value of x at t_{k+1} .

difference equations

This approximation¹ can be used in place of all derivatives that appear in the controller differential equations to arrive at a set of equations that can be solved by a digital computer. These equations are called **difference equations** and are solved repetitively with time steps of length T . For systems having bandwidths

¹ This particular version is called the **forward rectangular rule**. See Problem 3.2 for the **backward rectangular** version.

of a few Hertz, sample rates are often on the order of 100 Hz, so that sample periods are on the order of 10 msec and errors from the approximation can be quite small.

◆ Example 3.1 *Difference Equations Using Euler's Method*

Using Euler's method, find the difference equations to be programmed into the control computer in Fig. 3.1(b) for the case where the $D(s)$ in Fig. 3.1(a) is

$$D(s) = \frac{U(s)}{E(s)} = K_o \frac{s + a}{s + b} \quad (3.3)$$

Solution. First find the differential equation that corresponds to $D(s)$. After cross multiplying Eq. (3.3) to obtain

$$(s + b)U(s) = K_o(s + a)E(s),$$

we can see by inspection that the corresponding differential equation is

$$\dot{u} + bu = K_o(\dot{e} + ae). \quad (3.4)$$

Using Euler's method to approximate Eq. (3.4) according to Eq. (3.2), we get the approximating difference equation

$$\frac{u(k+1) - u(k)}{T} + bu(k) = K_o \left[\frac{e(k+1) - e(k)}{T} + ae(k) \right]. \quad (3.5)$$

Rearranging Eq. (3.5) puts the difference equation in the desired form

$$u(k+1) = u(k) + T \left[-bu(k) + K_o \left(\frac{e(k+1) - e(k)}{T} + ae(k) \right) \right]. \quad (3.6)$$

Equation (3.6) shows how to compute the new value of the control, $u(k+1)$, given the past value of the control, $u(k)$, and the new and past values of the error signal, $e(k+1)$ and $e(k)$. For computational efficiency, it is convenient to re-arrange Eq. (3.6) to

$$u(k+1) = (1 - bT)u(k) + K_o(aT - 1)e(k) + K_o e(k+1). \quad (3.7)$$

In principle, the difference equation is evaluated initially with $k = 0$, then $k = 1, 2, 3, \dots$. However, there is usually no requirement that values for all times be saved in memory. Therefore, the computer need only have variables defined for the current and past values for this first-order difference equation. The instructions to the computer to implement the feedback loop in Fig. 3.1(b) with the difference equation from Eq. (3.7) would call for a continual looping through the code in Table 3.1. Note in the table that the calculations have been arranged so as to minimize the computations required between the reading of the A/D and the writing to the D/A, thus keeping the computation delay to a minimum.

Table 3.1 Real Time Controller Implementation

```

x = 0 (initialization of past values for first loop through)
Define constants:
 $\alpha_1 = 1 - bT$ 
 $\alpha_2 = K_o(aT - 1)$ 
READ A/D to obtain y and r
e = r - y
u = x +  $K_o e$ 
OUTPUT u to D/A and ZOH
now compute x for the next loop through
 $x = \alpha_1 u + \alpha_2 e$ 
go back to READ when T seconds have elapsed since last READ

```

The sample rate required depends on the closed-loop bandwidth of the system. Generally, sample rates should be faster than 30 times the bandwidth in order to assure that the digital controller can be made to closely match the performance of the continuous controller. Discrete design methods described in later chapters will show how to achieve this performance and the consequences of sampling even slower if that is required for the computer being used. However, when using the techniques presented in this chapter, a good match to the continuous controller is obtained when the sample rate is greater than approximately 30 times the bandwidth.

◆ **Example 3.2** Lead Compensation Using a Digital Computer

Find digital controllers to implement the lead compensation

$$D(s) = 70 \frac{s + 2}{s + 10} \quad (3.8)$$

for the plant

$$G(s) = \frac{1}{s(s + 1)}$$

using sample rates of 20 Hz and 40 Hz. Implement the control equations on an experimental laboratory facility like that depicted in Fig. 3.1, that is, one that includes a microprocessor for the control equations, a ZOH, and analog electronics for the plant. Compute the theoretical step response of the continuous system and compare that with the experimentally determined step response of the digitally controlled system.

Solution. Comparing the compensation transfer function in Eq. (3.8) with Eq. (3.3) shows that the values of the parameters in Eq. (3.6) are $a = 2$, $b = 10$, and $K_o = 70$. For a sample rate of 20 Hz, $T = 0.05$ sec and Eq. (3.6) can be simplified to

$$u(k + 1) = 0.5u(k) + 70[e(k + 1) - 0.9e(k)].$$

For a sample rate of 40 Hz, $T = 0.025$ sec and Eq. (3.6) simplifies to

$$u(k+1) = 0.75u(k) + 70[e(k+1) - 0.95e(k)].$$

The statements in MATLAB to compute the continuous step response is

```
numD = 70*[1 2], denD = [1 10]
```

```
numG = 1, denG = [1 1 0]
```

```
sys1 = tf(numD,denD)*tf(numG,denG)
```

```
sysCL = feedback(sys1,1)
```

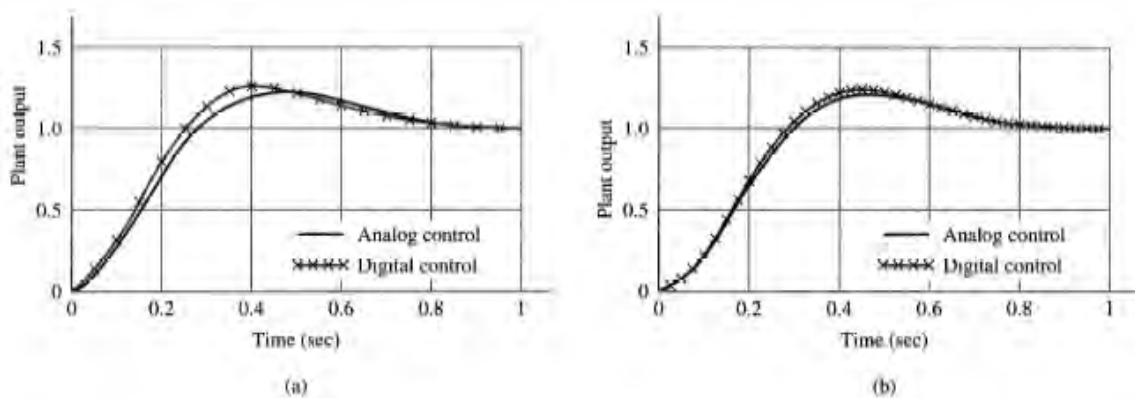
```
step(sysCL).
```

Figure 3.2 shows the step response of the two digital controllers compared to the continuous step response. Note that the 40 Hz sample rate (about $30 \times$ bandwidth) behaves essentially like the continuous case, whereas the 20 Hz sample rate (about $15 \times$ bandwidth) has a detectable increased overshoot signifying some degradation in the damping. The damping would degrade further if the sample rate were made any slower.

The MATLAB file that created Fig. 3.2 (fig32.m) computed the digital responses as well as the continuous response. You will learn how to compute the response of a digital system in Chapter 4.

Figure 3.2

Continuous and digital step response using Euler's method for discretization: (a) 20 Hz sample rate, (b) 40 Hz sample rate



In Chapter 6, you will see that there are several ways to approximate a continuous transfer function, each with different merits, and most with better qualities than the Euler method presented here. In fact, MATLAB provides a function (c2d.m) that computes these approximations. However, before those methods can be examined, it will be necessary to understand discrete transfer functions, a topic covered in Chapter 4.

3.2 Effect of Sampling

It is worthy to note that *the single most important* impact of implementing a control system digitally is the delay associated with the hold. A delay in any feedback system degrades the stability and damping of the system. Because each value of $u(kT)$ in Fig. 3.1(b) is held constant until the next value is available from the computer, the continuous value of $u(t)$ consists of steps (see Fig. 3.3) that, on the average, lag $u(kT)$ by $T/2$, as shown by the dashed line in the figure. By incorporating a continuous approximation of this $T/2$ delay in a continuous analysis of the system, an assessment can be made of the effect of the delay in the digitally controlled system. The delay can be approximated by the method of Padé. The simplest first-order approximation is

$$G_h(s) = \frac{2/T}{s + 2/T}. \quad (3.9)$$

Figure 3.4 compares the responses from Fig. 3.2 with a continuous analysis that includes a delay approximation according to Eq. (3.9).

This linear approximation of the sampling delay (Eq. (3.9)) could also be used to determine the effect of a particular sample rate on the roots of a system via linear analysis, perhaps a locus of roots vs. T . Alternatively, the effect of a delay can be analyzed using frequency response techniques because a time delay of $T/2$ translates into a phase decrease of

$$\delta\phi = -\frac{\omega T}{2}. \quad (3.10)$$

Figure 3.3
The delay due to the hold operation

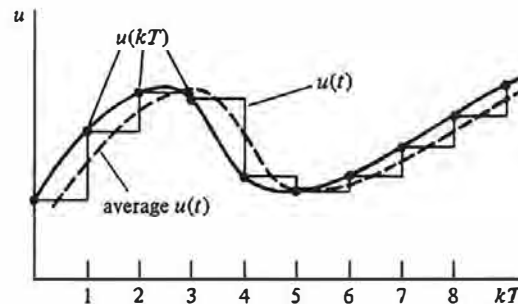
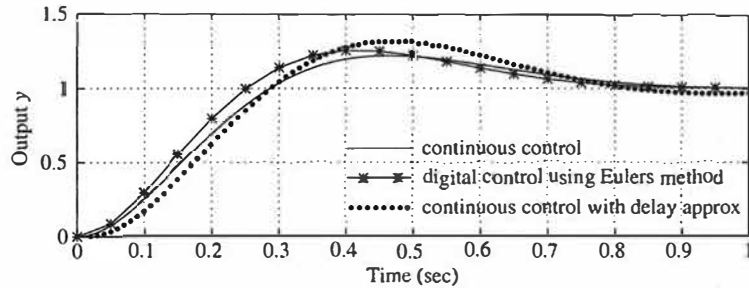


Figure 3.4

Continuous and digital step response at 20 Hz sample rate showing results with a $T/2$ delay approximation



Thus, we see that the loss of phase margin due to sampling can be estimated by invoking Eq. (3.10) with ω equal to the frequency where the magnitude equals one, that is, the “gain crossover frequency.”

◆ Example 3.3 *Approximate Analysis of the Effect of Sampling*

For the system in Example 3.2, determine the decrease in damping that would result from sampling at 10 Hz. Use both linear analysis and the frequency response method. Compare the time response of the continuous system with the discrete implementation to validate the analysis.

Solution. The damping of the system in Example 3.2 can be obtained from the MATLAB statement

```
damp(sysCL)
```

where sysCL is that computed in Example 3.2. The result is $\zeta = .56$.

The damping of the system with the simple delay approximation added (Eq. (3.9)) is obtained from

```
T = 1/10
```

```
numDL = 2/T, denDL = [1 2/T]
```

```
sys2 = tf(numDL,denDL)*sys1
```

```
sysCL = feedback(sys2,1)
```

```
damp(sysCL)
```

where sys1 is that computed in Example 3.2. The result of this calculation is $\zeta = .33$.

The frequency response of the continuous system is shown by the solid line in Fig. 3.5 and shows that the crossover frequency is about 6 rad/sec and the PM is about 50° . The line of small circles shows the phase corrected by Eq. (3.10) and, therefore, that the PM decreases to about 30° . For more precision, the use of `margin.m` in MATLAB shows that the continuous system has a PM of 49.5° at a crossover frequency of 6.17 rad/sec. Equation (3.10) then indicates that the correction due to sampling should be 17.7° , thus the PM of the digital system would be

31.8° . Since the PM is approximately $100 \times \zeta$, this analysis shows that the ζ decreases from approximately 0.5 for the continuous system to 0.32 for the digital system.

Both analysis methods indicate a similar reduction in the damping of the system. One should, therefore, expect that the overshoot of the step response should increase. For the case with no zeros, Fig. 2.7 indicates that this decrease in ζ should result in the step response overshoot, M_p , going from 16% to 35% for a 2nd-order system with no zeros. The actual step responses in Fig. 3.6 have about 20% overshoot for the continuous system and about 30% for the digital case. So, we see that the approximate analysis was somewhat conservative in the prediction of the decreased damping and increased overshoot in the digital case. The trend that decreasing sample rate causes decreasing damping and stability will be analyzed more completely throughout the book.

Figure 3.5
Frequency response for
Example 3.3

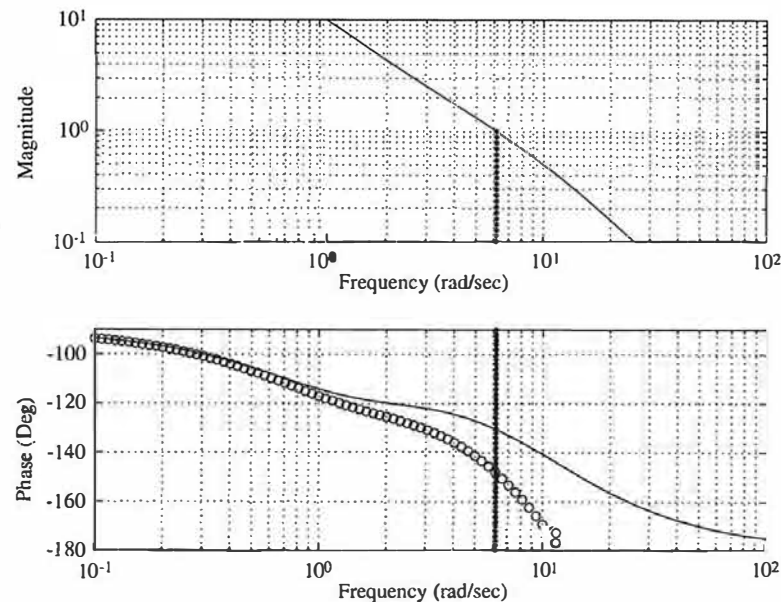
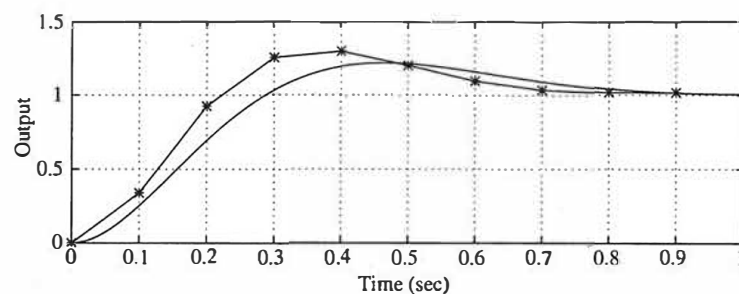


Figure 3.6
Continuous and digital
responses for Example
3.3 (at 10 Hz sample
rate)



3.3 PID Control

The notion of proportional, integral, and derivative (PID) control is reviewed in Section 2.2.3. Reviewing again briefly, the three terms are proportional control

$$u(t) = Ke(t), \quad (3.11)$$

integral control

$$u(t) = \frac{K}{T_i} \int_0^t e(\eta) d\eta, \quad (3.12)$$

and derivative control

$$u(t) = KT_d \dot{e}(t), \quad (3.13)$$

where K is called the proportional gain, T_i the integral time, and T_d the derivative time. These three constants define the control.

The approximations of these individual control terms to an algebraic equation that can be implemented in a digital computer are proportional control

$$u(k) = Ke(k), \quad (3.14)$$

integral control

$$u(k) = u(k-1) + \frac{K}{T_i} Te(k), \quad (3.15)$$

and derivative control

$$u(k) = \frac{KT_d}{T} [e(k) - e(k-1)]. \quad (3.16)$$

Equation (3.11) is already algebraic, therefore Eq. (3.14) follows directly while Eqs. (3.15) and (3.16) result from an application of Euler's method (Eq. (3.2)) to Eqs. (3.12) and (3.13). However, normally these terms are used together and, in this case, the combination needs to be done carefully. The combined continuous transfer function (Eq. 2.24) is

$$D(s) = \frac{u(s)}{e(s)} = K \left(1 + \frac{1}{T_i s} + T_d s \right).$$

Therefore, the differential equation relating $u(t)$ and $e(t)$ is

$$\dot{u} = K \left(\dot{e} + \frac{1}{T_i} e + T_d \ddot{e} \right)$$

and the use of Euler's method (twice for $\ddot{e}$) results in

$$u(k) = u(k-1) + K \left[\left(1 + \frac{T}{T_i} + \frac{T_d}{T} \right) e(k) - \left(1 + 2\frac{T_d}{T} \right) e(k-1) + \frac{T_d}{T} e(k-2) \right]. \quad (3.17)$$

◆ **Example 3.4** *Transforming a Continuous PID to a Digital Computer*

A micro-servo motor has a transfer function from the input applied voltage to the output speed (rad/sec),

$$G(s) = \frac{360000}{(s + 60)(s + 600)}. \quad (3.18)$$

It has been determined that PID control with $K = 5$, $T_D = 0.0008$ sec, and $T_I = 0.003$ sec gives satisfactory performance for the continuous case. Pick an appropriate sample rate, determine the corresponding digital control law, and implement on a digital system. Compare the digital step response with the calculated response of a continuous system. Also, separately investigate the effect of a higher sample rate and re-tuning the PID parameters on the ability of the digital system to match the continuous response.

Solution. The sample rate needs to be selected first. But before we can do that, we need to know how fast the system is or what its bandwidth is. The solid line in Fig. 3.7 shows the step response of the continuous system and indicates that the rise time is about 1 msec. Based on Eq. (2.16), this suggests that $\omega_n \cong 1800$ rad/sec, and so the bandwidth would be on the order of 2000 rad/sec or 320 Hz. Therefore, the sample rate would be about 3.2 kHz if 10 times bandwidth. So let's pick $T = 0.3$ msec. Use of Eq. (3.17) results in the difference equation

$$u(k) = u(k - 1) + 5[3.7667e(k) - 6.3333e(k - 1) + 2.6667e(k - 2)]$$

which, when implemented in the digital computer results in the line with stars in Fig. 3.7. This implementation shows a considerably increased overshoot over the continuous case. The line with circles in the figure shows the improved performance obtained by increasing the sample rate to 10 kHz; i.e., a sample rate about 30 times bandwidth, while using the same PID parameters as before. It shows that the digital performance has improved to be essentially the same as the continuous case.

Increasing the sample rate, however, will increase the cost of the computer and the A/D converter; therefore, there will be a cost benefit by improving the performance while maintaining the 3.2 kHz sample rate. A look at Fig. 3.7 shows that the digital response ($T = 0.3$ msec) has a faster rise time and less damping than the continuous case. This suggests that the proportional gain, K , should be reduced to slow the system down and the derivative time, T_D , should be increased to increase the damping. Some trial and error, keeping these ideas in mind, produces the results in Fig. 3.8. The revised PID parameters that produced these results are $K = 3.2$ and $T_D = 0.0011$ sec. The integral reset time, T_I , was left unchanged.

This example once again showed the characteristics of a digital control system. The damping was degraded an increasing amount as the sample rate was reduced. Furthermore, it was possible to restore the damping with suitable adjustments to the control.

Figure 3.7
Step response of a
micro-motor, Example
3.4, same PID
parameters

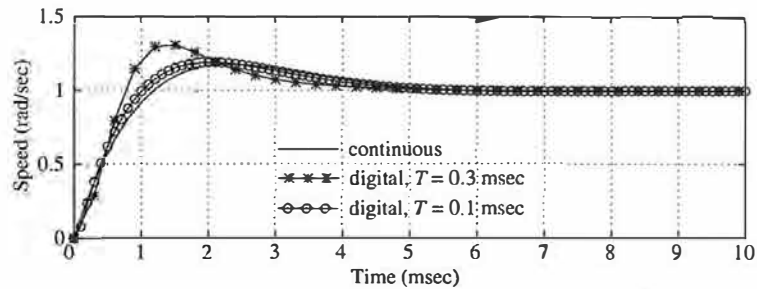
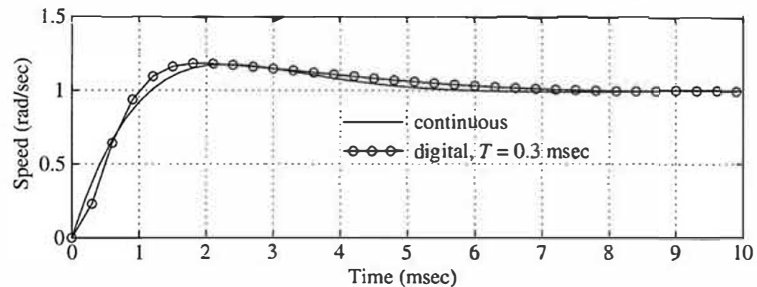


Figure 3.8
Effect of PID tuning on
the digital response,
Example 3.4



3.4 Summary

- Digitization methods allow the designer to convert a continuous compensation, $D(s)$, into a set of difference equations that can be programmed directly into a control computer.
- Euler's method can be used for the digitization

$$\dot{x}(k) \cong \frac{x(k+1) - x(k)}{T} \quad (3.2)$$

- As long as the sample rate is on the order of $30 \times$ bandwidth or faster, the digitally controlled system will behave close to its continuous counterpart and the continuous analysis that has been the subject of your continuous control systems study will suffice.
- For sample rates on the order of 10 to 30 times the bandwidth, a first order analysis can be carried out by introducing a delay of $T/2$ in the continuous analysis to see how well the digital implementation matches the continuous analysis. A zero-pole approximation for this delay is

$$G_h(s) = \frac{2/T}{s + 2/T} \quad (3.9)$$

The delay can be analyzed more accurately using frequency response where the phase from the continuous analysis should be decreased by

$$\delta\phi = \frac{\omega T}{2}. \quad (3.10)$$

- A continuous PID control law whose transfer function is

$$D(s) = \frac{u(s)}{e(s)} = K \left(1 + \frac{1}{T_i s} + T_d s \right)$$

can be implemented digitally using Eq. (3.17)

$$u(k) = u(k-1) + K \left[\left(1 + \frac{T}{T_i} + \frac{T_d}{T} \right) e(k) - \left(1 + 2\frac{T_d}{T} \right) e(k-1) + \frac{T_d}{T} e(k-2) \right].$$

The digital control system will behave reasonably close to the continuous system providing the sample rate is faster than 30 times the bandwidth.

- In order to analyze the system accurately for any sample rate, but especially for sample rates below about 30 times bandwidth, you will have to proceed on to the next chapters to learn about z-transforms and how to apply them to the study of discrete systems.
- For digital control systems with sample rates less than 30 times bandwidth, design is often carried out directly in the discrete domain, eliminating approximation errors.

3.5 Problems

3.1 Do the following:

- Design a continuous lead compensation for the satellite attitude control example ($G(s) = 1/s^2$) described in Appendix A.1 so that the complex roots are at approximately $s = -4.4 \pm j4.4$ rad/sec.
- Assuming the compensation is to be implemented digitally, approximate the effect of the digital implementation to be a delay of $T/2$ as given by

$$G_h(s) = \frac{2/T}{s + 2/T}$$

and determine the revised root locations for sample rates of $\omega_s = 5$ Hz, 10 Hz, and 20 Hz where $T = 1/\omega_s$ sec.

3.2 Repeat Example 3.1 (from page 60), but use the approximation that

$$\dot{x}(k) \cong \frac{x(k) - x(k-1)}{T},$$

the **backward rectangular** version of Euler's method. Compare the resulting difference equations with the forward rectangular Euler method. Also compute the numerical value of the coefficients for both cases vs. sample rate for $\omega_s = 1 - 100$ Hz. Assume the continuous values from Eq. (3.8). The three coefficients of interest for both the forward and backward rectangular cases are the coefficients of $u(k)$, $e(k)$, and $e(k+1)$.

3.3 For the compensation

$$D(s) = 25 \frac{s+1}{s+15},$$

use Euler's forward rectangular method to determine the difference equations for a digital implementation with a sample rate of 80 Hz. Repeat the calculations using the backward rectangular method (see Problem 3.2) and compare the difference equation coefficients.

3.4 For the compensation

$$D(s) = 5 \frac{s+2}{s+20},$$

use Euler's forward rectangular method to determine the difference equations for a digital implementation with a sample rate of 80 Hz. Repeat the calculations using the backward rectangular method (see Problem 3.2) and compare the difference equation coefficients.

3.5 The read arm on a computer disk drive has the transfer function

$$G(s) = \frac{1000}{s^2}.$$

Design a digital PID controller that has a bandwidth of 100 Hz, a phase margin of 50°, and has no output error for a constant bias torque from the drive motor. Use a sample rate of 6 kHz.

3.6 The read arm on a computer disk drive has the transfer function

$$G(s) = \frac{1000}{s^2}.$$

Design a digital controller that has a bandwidth of 100 Hz and a phase margin of 50°. Use a sample rate of 6 kHz.

3.7 For

$$G(s) = \frac{1}{s^2},$$

- design a continuous compensation so that the closed-loop system has a rise time $t_r < 1$ sec and overshoot $M_p < 15\%$ to a step input command,
- revise the compensation so the specifications would still be met if the feedback was implemented digitally with a sample rate of 5 Hz, and
- find difference equations that will implement the compensation in the digital computer.

3.8 The read arm on a computer disk drive has the transfer function

$$G(s) = \frac{500}{s^2}.$$

Design a continuous lead compensation so that the closed-loop system has a bandwidth of 100 Hz and a phase margin of 50°. Modify the MATLAB file fig32.m so that you can evaluate the digital version of your lead compensation using Euler's forward rectangular method. Try different sample rates, and find the slowest one where the overshoot does not exceed 30%.

3.9 The antenna tracker has the transfer function

$$G(s) = \frac{10}{s(s+2)}.$$

Design a continuous lead compensation so that the closed-loop system has a rise time $t_r < 0.3$ sec and overshoot $M_p < 10\%$. Modify the MATLAB file fig32.m so that you can evaluate the digital version of your lead compensation using Euler's forward rectangular method. Try different sample rates, and find the slowest one where the overshoot does not exceed 20%.

3.10 The antenna tracker has the transfer function

$$G(s) = \frac{10}{s(s+2)}.$$

Design a continuous lead compensation so that the closed-loop system has a rise time $t_r < 0.3$ sec and overshoot $M_p < 10\%$. Approximate the effect of a digital implementation to be

$$G_h(s) = \frac{2/T}{s + 2/T},$$

and estimate M_p for a digital implementation with a sample rate of 10 Hz.

♦ 4 ♦

Discrete Systems Analysis

A Perspective on Discrete Systems Analysis

The unique element in the structure of Fig. 3.1 is the digital computer. The fundamental character of the digital computer is that it takes a finite time to compute answers, and it does so at discrete steps in time. The purpose of this chapter is to develop tools of analysis necessary to understand and to guide the design of programs for a computer sampling at discrete times and acting as a linear, dynamic control component. Needless to say, digital computers can do many things other than control linear dynamic systems; it is our purpose in this chapter to examine their characteristics when doing this elementary control task and to develop the basic analysis tools needed to write programs for real-time computer control.

Chapter Overview

Section 4.1 restates the difference equations used by a computer to represent a dynamic system, a topic covered very briefly in Section 3.1. The tool for analyzing this sort of system, the z -transform, is introduced and developed in Section 4.2. Use of the z -transform is developed further in Section 4.3 to show how it applies to the combined system in Fig. 3.1. Furthermore, state-space models of discrete systems are developed in this section. Section 4.4 shows the correspondence between roots in the z -plane and time response characteristics while Section 4.5 discusses characteristics of the discrete frequency response. The last section, 4.6, derives properties of the z -transform.

4.1 Linear Difference Equations

We assume that the analog-to-digital converter (A/D) in Fig. 1.1 takes samples of the signal y at discrete times and passes them to the computer so that $\hat{y}(kT) =$

$y(kT)$. The job of the computer is to take these sample values and compute in some fashion the signals to be put out through the digital-to-analog converter (D/A). The characteristics of the A/D and D/A converters will be discussed later. Here we consider the treatment of the data inside the computer. Suppose we call the input signals up to the k th sample $e_0, e_1, e_2, \dots, e_k$, and the output signals prior to that time $u_0, u_1, u_2, \dots, u_{k-1}$. Then, to get the next output, we have the machine compute some function, which we can express in symbolic form as

$$u_k = f(e_0, \dots, e_k; u_0, \dots, u_{k-1}). \quad (4.1)$$

Because we plan to emphasize the elementary and the dynamic possibilities, we assume that the function f in Eq. (4.1) is *linear* and depends on only a *finite* number of past e 's and u 's. Thus we write

$$u_k = -a_1 u_{k-1} - a_2 u_{k-2} - \dots - a_n u_{k-n} + b_0 e_k + b_1 e_{k-1} + \dots + b_m e_{k-m}. \quad (4.2)$$

Equation (4.2) is called a linear **recurrence equation** or **difference equation** and, as we shall see, has many similarities with a linear differential equation. The name "difference equation" derives from the fact that we could write Eq. (4.2) using u_k plus the differences in u_k , which are defined as

$$\begin{aligned} \nabla u_k &= u_k - u_{k-1} \\ \nabla^2 u_k &= \nabla u_k - \nabla u_{k-1} \\ \nabla^n u_k &= \nabla^{n-1} u_k - \nabla^{n-1} u_{k-1}. \end{aligned} \quad (4.3)$$

If we solve Eq. (4.3) for the values of u_k , u_{k-1} , and u_{k-2} in terms of differences, we find

$$\begin{aligned} u_k &= u_k, \\ u_{k-1} &= u_k - \nabla u_k \\ u_{k-2} &= u_k - 2\nabla u_k + \nabla^2 u_k. \end{aligned}$$

Thus, for a second-order equation with coefficients a_1, a_2 , and b_0 (we let $b_1 = b_2 = 0$ for simplicity), we find the equivalent difference equation to be

$$a_2 \nabla^2 u_k - (a_1 + 2a_2) \nabla u_k + (a_2 + a_1 + 1) u_k = b_0 e_k.$$

Although the two forms are equivalent, the recurrence form of Eq. (4.2) is more convenient for computer implementation; we will drop the form using differences. We will continue, however, to refer to our equations as "difference equations." If the a 's and b 's in Eq. (4.2) are constant, then the computer is solving a **constant-coefficient difference equation** (CCDE). We plan to demonstrate later that with such equations the computer can control linear constant dynamic systems and approximate most of the other tasks of linear, constant, dynamic systems, including performing the functions of electronic filters. To do so, it is necessary first to examine methods of obtaining solutions to Eq. (4.2) and to study the general properties of these solutions.

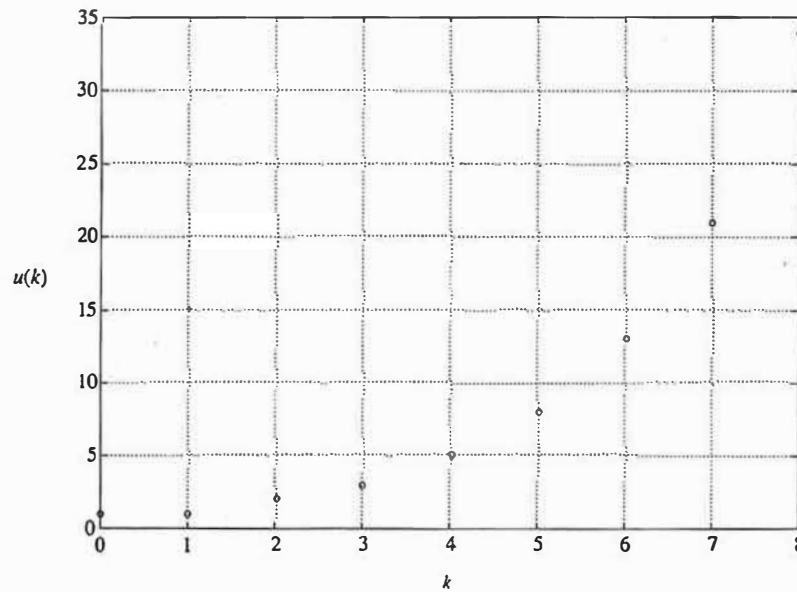
To solve a specific CCDE is an elementary matter. We need a starting time (k -value) and some initial conditions to characterize the contents of the computer memory at this time. For example, suppose we take the case

$$u_k = u_{k-1} + u_{k-2} \quad (4.4)$$

and start at $k = 2$. Here there are no input values, and to compute u_2 we need to know the (initial) values for u_0 and u_1 . Let us take them to be $u_0 = u_1 = 1$. The first nine values are 1, 1, 2, 3, 5, 8, 13, 21, 34, ... A plot of the values of u_k versus k is shown in Fig. 4.1.

The results, the **Fibonacci numbers**, are named after the thirteenth-century mathematician¹ who studied them. For example, Eq. (4.4) has been used to model the growth of rabbits in a protected environment². However that may be, the output of the system represented by Eq. (4.4) would seem to be growing, to say the least. If the response of a dynamic system to any finite initial conditions can grow without bound, we call the system **unstable**. We would like to be able to examine equations like Eq. (4.2) and, without having to solve them explicitly, see if they are stable or unstable and even understand the general shape of the solution.

Figure 4.1
The Fibonacci numbers



¹ Leonardo Fibonacci of Pisa, who introduced Arabic notation to the Latin world about 1200 A.D.

² Wilde (1964). Assume that u_k represents pairs of rabbits and that babies are born in pairs. Assume that no rabbits die and that a new pair begin reproduction after one period. Thus at time k , we have all the old rabbits, u_{k-1} , plus the newborn pairs born to the mature rabbits, which are u_{k-2} .

One approach to solving this problem is to assume a form for the solution with unknown constants and to solve for the constants to match the given initial conditions. For continuous, ordinary, differential equations that are constant and linear, exponential solutions of the form e^{st} are used. In the case of linear, constant, difference equations, it turns out that solutions of the form z^k will do where z has the role of s and k is the discrete independent variable replacing time, t . Consider Eq. (4.4). If we assume that $u(k) = Az^k$, we get the equation

$$Az^k = Az^{k-1} + Az^{k-2}.$$

Now if we assume $z \neq 0$ and $A \neq 0$, we can divide by A and multiply by z^{-k} , with the result

$$1 = z^{-1} + z^{-2}$$

or

$$z^2 = z + 1.$$

This polynomial of second degree has two solutions, $z_{1,2} = 1/2 \pm \sqrt{5}/2$. Let's call these z_1 and z_2 . Since our equation is linear, a sum of the individual solutions will also be a solution. Thus, we have found that a solution to Eq. (4.4) is of the form

$$u(k) = A_1 z_1^k + A_2 z_2^k.$$

We can solve for the unknown constants by requiring that this general solution satisfy the specific initial conditions given. If we substitute $k = 0$ and $k = 1$, we obtain the simultaneous equations

$$\begin{aligned} 1 &= A_1 + A_2, \\ 1 &= A_1 z_1 + A_2 z_2. \end{aligned}$$

These equations are easily solved to give

$$\begin{aligned} A_1 &= \frac{\sqrt{5} + 1}{2\sqrt{5}}, \\ A_2 &= \frac{\sqrt{5} - 1}{2\sqrt{5}}. \end{aligned}$$

And now we have the complete solution of Eq. (4.4) in a closed form. Furthermore, we can see that since $z_1 = (1 + \sqrt{5})/2$ is greater than 1, the term in z_1^k will grow without bound as k grows, which confirms our suspicion that the equation represents an unstable system. We can generalize this result. The equation in z that we obtain after we substitute $u = z^k$ is a polynomial in z known as the **characteristic equation** of the difference equation. If any solution of this equation is outside the unit circle (has a magnitude greater than one),

characteristic equation

the corresponding difference equation is unstable in the specific sense that for some finite initial conditions the solution will grow without bound as time goes to infinity. If *all* the roots of the characteristic equation are *inside* the unit circle, the corresponding difference equation is stable.

◆ **Example 4.1** *Discrete Stability*

Is the equation

$$u(k) = 0.9u(k-1) - 0.2u(k-2)$$

stable?

Solution. The characteristic equation is

$$z^2 - 0.9z + 0.2 = 0,$$

and the characteristic roots are $z = 0.5$ and $z = 0.4$. Since both these roots are inside the unit circle, the equation is stable.

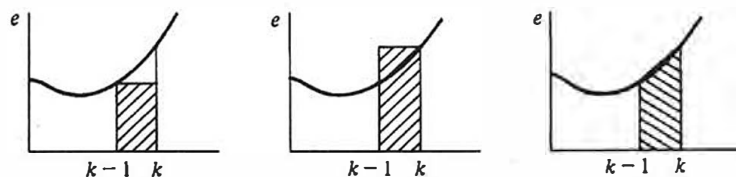
As an example of the origins of a difference equation with an external input, we consider the discrete approximation to integration. Suppose we have a continuous signal, $e(t)$, of which a segment is sketched in Fig. 4.2, and we wish to compute an approximation to the integral

$$\mathcal{J} = \int_0^t e(t) dt, \quad (4.5)$$

using only the discrete values $e(0), \dots, e(t_{k-1}), e(t_k)$. We assume that we have an approximation for the integral from zero to the time t_{k-1} and we call it u_{k-1} . The problem is to obtain u_k from this information. Taking the view of the integral as the area under the curve $e(t)$, we see that this problem reduces to finding an approximation to the area under the curve between t_{k-1} and t_k . Three alternatives are sketched in Fig. 4.2. We can use the rectangle of height e_{k-1} , or the rectangle

Figure 4.2

Plot of a function and alternative approximations to the area under the curve over a single time interval



of height e_k , or the trapezoid formed by connecting e_{k-1} to e_k by a straight line. If we take the third choice, the area of the trapezoid is

$$A = \frac{t_k - t_{k-1}}{2}(e_k + e_{k-1}). \quad (4.6)$$

Trapezoid rule

Finally, if we assume that the sampling period, $t_k - t_{k-1}$, is a constant, T , we are led to a simple formula for discrete integration called the **trapezoid rule**

$$u_k = u_{k-1} + \frac{T}{2}(e_k + e_{k-1}). \quad (4.7)$$

If $e(t) = t$, then $e_k = kT$ and substitution of $u_k = (T^2/2)k^2$ satisfies Eq. (4.7) and is exactly the integral of e . (It should be, because if $e(t)$ is a straight line, the trapezoid is the *exact* area.) If we approximate the area under the curve by the rectangle of height e_{k-1} , the result is called the **forward rectangular rule** (sometimes called Euler's method, as discussed in Chapter 3 for an approximation to differentiation) and is described by

$$u_k = u_{k-1} + Te_{k-1}.$$

The other possibility is the **backward rectangular rule**, given by

$$u_k = u_{k-1} + Te_k.$$

Each of these integration rules is a special case of our general difference equation Eq. (4.2). We will examine the properties of these rules later, in Chapter 6, while discussing means to obtain a difference equation that will be equivalent to a given differential equation.

Thus we see that difference equations can be evaluated directly by a digital computer and that they can represent models of physical processes and approximations to integration. It turns out that if the difference equations are linear with coefficients that are constant, we can describe the relation between u and e by a transfer function, and thereby gain a great aid to analysis and also to the design of linear, constant, discrete controls.

4.2 The Discrete Transfer Function

We will obtain the transfer function of linear, constant, discrete systems by the method of z -transform analysis. A logical alternative viewpoint that requires a bit more mathematics but has some appeal is given in Section 4.6.2. The results are the same. We also show how these same results can be expressed in the state space form in Section 4.2.3.

4.2.1 The z -Transform

If a signal has discrete values $e_0, e_1, \dots, e_k, \dots$ we define the z -transform of the signal as the function^{3,4}

$$\begin{aligned} E(z) &\triangleq \mathcal{Z}\{e(k)\} \\ &\triangleq \sum_{k=-\infty}^{\infty} e_k z^{-k}, \quad r_o < |z| < R_o, \end{aligned} \quad (4.8)$$

and we assume we can find values of r_o and R_o as bounds on the magnitude of the complex variable z for which the series Eq. (4.8) converges. A discussion of convergence is deferred until Section 4.6.

◆ Example 4.2 The z -Transform

The data e_k are taken as samples from the time signal $e^{-at} \mathbf{1}(t)$ at sampling period T where $\mathbf{1}(t)$ is the unit step function, zero for $t < 0$, and one for $t \geq 0$. Then $e_k = e^{-akT} \mathbf{1}(kT)$. Find the z -transform of this signal.

Solution. Applying Eq. (4.8), we find that

$$\begin{aligned} \sum_{k=-\infty}^{\infty} e_k z^{-k} &= \sum_0^{\infty} e^{-akT} z^{-k} \\ &= \sum_0^{\infty} (e^{-aT} z^{-1})^k \\ &= \frac{1}{1 - e^{-aT} z^{-1}} \\ &= \frac{z}{z - e^{-aT}} \quad |z| > e^{-aT}. \end{aligned}$$

We will return to the analysis of signals and development of a table of useful z -transforms in Section 4.4; we first examine the use of the transform to reduce

³ We use the notation $\triangleq$ to mean "is defined as."

⁴ In Eq. (4.8) the lower limit is $-\infty$ so that values of e_k on both sides of $k = 0$ are included. The transform so defined is sometimes called the two-sided z -transform to distinguish it from the one-sided definition, which would be $\sum_0^{\infty} e_k z^{-k}$. For signals that are zero for $k < 0$, the transforms obviously give identical results. To take the one-sided transform of u_{k-1} , however, we must handle the value of u_{-1} , and thus are initial conditions introduced by the one-sided transform. Examination of this property and other features of the one-sided transform are invited by the problems. We select the two-sided transform because we need to consider signals that extend into negative time when we study random signals in Chapter 12.

difference equations to algebraic equations and techniques for representing these as block diagrams.

4.2.2 The Transfer Function

The z -transform has the same role in discrete systems that the Laplace transform has in analysis of continuous systems. For example, the z -transforms for e_k and u_k in the difference equation (4.2) or in the trapezoid integration (4.7) are related in a simple way that permits the rapid solution of linear, constant, difference equations of this kind. To find the relation, we proceed by direct substitution. We take the definition given by Eq. (4.8) and, in the same way, we define the z -transform of the sequence $\{u_k\}$ as

$$U(z) = \sum_{k=-\infty}^{\infty} u_k z^{-k}. \quad (4.9)$$

Now we multiply Eq. (4.7) by z^{-k} and sum over k . We get

$$\sum_{k=-\infty}^{\infty} u_k z^{-k} = \sum_{k=-\infty}^{\infty} u_{k-1} z^{-k} + \frac{T}{2} \left(\sum_{k=-\infty}^{\infty} e_k z^{-k} + \sum_{k=-\infty}^{\infty} e_{k-1} z^{-k} \right). \quad (4.10)$$

From Eq. (4.9), we recognize the left-hand side as $U(z)$. In the first term on the right, we let $k - 1 = j$ to obtain

$$\sum_{k=-\infty}^{\infty} u_{k-1} z^{-k} = \sum_{j=-\infty}^{\infty} u_j z^{-(j+1)} = z^{-1} U(z). \quad (4.11)$$

By similar operations on the third and fourth terms we can reduce Eq. (4.10) to

$$U(z) = z^{-1} U(z) + \frac{T}{2} [E(z) + z^{-1} E(z)]. \quad (4.12)$$

Equation (4.12) is now simply an algebraic equation in z and the functions U and E . Solving it we obtain

$$U(z) = \frac{T}{2} \frac{1 + z^{-1}}{1 - z^{-1}} E(z). \quad (4.13)$$

We *define* the ratio of the transform of the output to the transform of the input as the **transfer function**, $H(z)$. Thus, in this case, the transfer function for trapezoid-rule integration is

$$\frac{U(z)}{E(z)} \triangleq H(z) = \frac{T}{2} \frac{z + 1}{z - 1}. \quad (4.14)$$

For the more general relation given by Eq. (4.2), it is readily verified by the same techniques that

$$H(z) = \frac{b_0 + b_1 z^{-1} + \dots + b_m z^{-m}}{1 + a_1 z^{-1} + a_2 z^{-2} + \dots + a_n z^{-n}},$$

and if $n \geq m$, we can write this as a ratio of polynomials in z as

$$H(z) = \frac{b_0 z^n + b_1 z^{n-1} + \cdots + b_m z^{n-m}}{z^n + a_1 z^{n-1} + a_2 z^{n-2} + \cdots + a_n} \quad (4.15)$$

or

$$H(z) = \frac{b(z)}{a(z)}.$$

This transfer function is represented in MATLAB in the tf form similarly to the continuous case as discussed after Eq. (2.6). The numerator of Eq. (4.15) would be specified in MATLAB as a $1 \times (n+1)$ matrix of the coefficients, for example, when $m = n$

$$\text{num} = [b_0 \quad b_1 \quad b_2 \quad \cdots \quad b_n]$$

and when $n > m$, there would be $n - m$ zeros after b_m . The quantity specifying the denominator would be specified as a $1 \times (n+1)$ matrix, for example

$$\text{den} = [1 \quad a_1 \quad a_2 \quad \cdots \quad a_n].$$

Note that $H(z)$ was assumed to be in the form given by Eq. (4.15), that is, with positive powers of z . The discrete system is specified as

$$\text{sys} = \text{tf}(\text{num}, \text{den}, T)$$

where T is the sample period.

The general input-output relation between transforms with linear, constant, difference equations is

$$U(z) = H(z)E(z). \quad (4.16)$$

Although we have developed the transfer function with the z -transform, it is also true that the transfer function is the ratio of the output to the input when both vary as z^k .

Because $H(z)$ is a rational function of a complex variable, we use the terminology of that subject. Suppose we call the numerator polynomial $b(z)$ and the denominator $a(z)$. The places in z where $b(z) = 0$ are **zeros** of the transfer function, and the places in z where $a(z) = 0$ are the **poles** of $H(z)$. If z_0 is a pole and $(z - z_0)^p H(z)$ has neither pole nor zero at z_0 , we say that $H(z)$ has a pole of order p at z_0 . If $p = 1$, the pole is simple. The transfer function Eq. (4.14) has a simple pole at $z = 1$ and a simple zero at $z = -1$. When completely factored, the transfer function would be

$$H(z) = K \frac{\prod_{i=1}^m (z - z_i)}{\prod_{i=1}^n (z - p_i)}, \quad (4.17)$$

zeros
poles

and the quantities specifying the transfer function in the MATLAB zpk form are an $m \times 1$ matrix of the zeros, an $n \times 1$ matrix of the poles, and a scalar gain, for example

$$z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{bmatrix}, \quad p = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix}, \quad k = K.$$

The system is then

$$\text{sys} = \text{zpk}(z, p, k, T).$$

We can now give a physical meaning to the variable z . Suppose we let all coefficients in Eq. (4.15) be zero except b_1 and we take b_1 to be 1. Then $H(z) = z^{-1}$. But $H(z)$ represents the transform of Eq. (4.2), and with these coefficient values the difference equation reduces to

$$u_k = e_{k-1}. \quad (4.18)$$

z^{-1} and cycle delay

The present value of the output, u_k , equals the input *delayed by one period*. Thus we see that a transfer function of z^{-1} is a *delay* of one time unit. We can picture the situation as in Fig. 4.3, where both time and transform relations are shown.

Since the relations of Eqs. (4.7), (4.14), (4.15) are all composed of delays, they can be expressed in terms of z^{-1} . Consider Eq. (4.7). In Fig. 4.4 we illustrate the difference equation (4.7) using the transfer function z^{-1} as the symbol for a unit delay.

We can follow the operations of the discrete integrator by tracing the signals through Fig. 4.4. For example, the present value of e_k is passed to the first summer, where it is added to the previous value e_{k-1} , and the sum is multiplied by $T/2$ to compute the area of the trapezoid between e_{k-1} and e_k . This is the signal marked a_k in Fig. 4.4. After this, there is another sum, where the previous output, u_{k-1} , is added to the new area to form the next value of the integral estimate, u_k . The discrete integration occurs in the loop with one delay, z^{-1} , and unity gain.

4.2.3 Block Diagrams and State-Variable Descriptions

Because Eq. (4.16) is a linear algebraic relationship, a system of such relations is described by a system of linear equations. These can be solved by the methods of linear algebra or by the graphical methods of block diagrams in the same

Figure 4.3
The unit delay

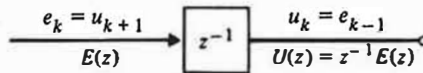
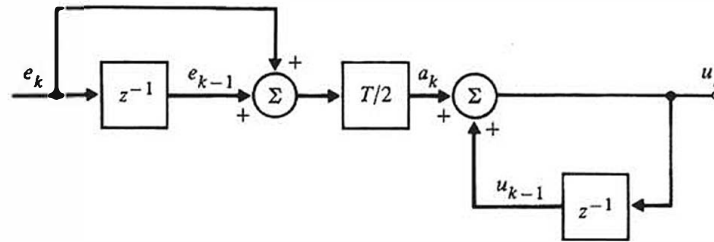


Figure 4.4

A block diagram of trapezoid integration as represented by Eq. (4.7)



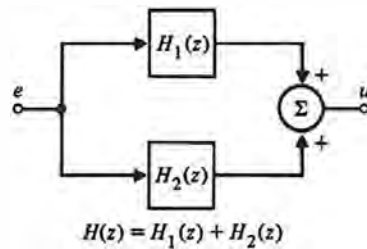
way as for continuous system transfer functions. To use block-diagram analysis to manipulate these discrete-transfer-function relationships, there are only four primitive cases:

1. The transfer function of paths in parallel is the sum of the single-path transfer functions (Fig. 4.5).
2. The transfer function of paths in series is the *product* of the path transfer functions (Fig. 4.6).
3. The transfer function of a single loop of paths is the transfer function of the forward path divided by one minus the loop transfer function (Fig. 4.7).
4. The transfer function of an arbitrary multipath diagram is given by combinations of these cases. Mason's rule⁶ can also be used.

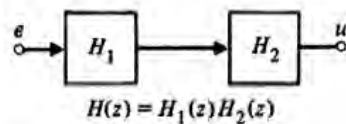
For the general difference equation of (4.2), we already have the transfer function in Eq. (4.15). It is interesting to connect this case with a block diagram

Figure 4.5

Block diagram of parallel blocks

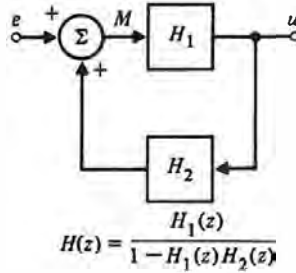
**Figure 4.6**

Block diagram of cascade blocks



⁶ Mason (1956). See Franklin, Powell, and Emami-Naeini (2019) for a discussion.

Figure 4.7
Feedback transfer
function



using only simple delay forms for z in order to see several “canonical” block diagrams and to introduce the description of discrete systems using equations of state.

*Canonical Forms

control canonical form

There are many ways to reduce the difference equation (4.2) to a block diagram involving z only as the delay operator, z^{-1} . The first one we will consider leads to the “control” canonical form. We begin with the transfer function as a ratio of polynomials

$$U(z) = H(z)E(z) = \frac{b(z)}{a(z)}E(z) = b(z)\xi,$$

where

$$\xi = \frac{E(z)}{a(z)}$$

and thus

$$a(z)\xi = E(z).$$

At this point we need to get specific; and rather than carry through with a system of arbitrary order, we will work out the details for the third-order case. In the development that follows, we will consider the variables u , e , and ξ as *time* variables and z as an advance operator such that $zu(k) = u(k+1)$ or $z^{-1}u(k) = u(k-1)$. With this convention (which is simply using the property of z derived earlier), consider the equations

$$(z^3 + a_1z^2 + a_2z + a_3)\xi = e, \quad (4.19)$$

$$(b_0z^3 + b_1z^2 + b_2z + b_3)\xi = u. \quad (4.20)$$

We can write Eq. (4.19) as

$$\begin{aligned} z^3\xi &= e - a_1z^2\xi - a_2z\xi - a_3\xi, \\ \xi(k+3) &= e(k) - a_1\xi(k+2) - a_2\xi(k+1) - a_3\xi(k). \end{aligned} \quad (4.21)$$

Now assume we have $z^3\xi$, which is to say that we have $\xi(k+3)$ because z^3 is an advance operator of three steps. If we operate on this with z^{-1} three times in a row, we will get back to $\xi(k)$, as shown in Fig. 4.8(a). From Eq. (4.21), we can now compute $z^3\xi$ from e and the lower powers of z and ξ given in the block diagram; the picture is now as given in Fig. 4.8(b). To complete the representation of Eqs. (4.19) and (4.20), we need only add the formation of the output u as a weighted sum of the variables $z^3\xi$, $z^2\xi$, $z\xi$, and ξ according to Eq. (4.20). The completed picture is shown in Fig. 4.8(c).

In Fig. 4.8(c), the internal variables have been named x_1 , x_2 , and x_3 . These variables comprise the *state* of this dynamic system in this form. Having the block diagram shown in Fig. 4.8(c), we can write down, almost by inspection, the difference equations that describe the evolution of the state, again using the fact that the transfer function z^{-1} corresponds to a one-unit delay. For example, we see that $x_3(k+1) = x_2(k)$ and $x_2(k+1) = x_1(k)$. Finally, expressing the sum at the far left of the figure, we have

$$x_1(k+1) = -a_1x_1(k) - a_2x_2(k) - a_3x_3(k) + e(k).$$

We collect these three equations together in proper order, and we have

$$\begin{aligned} x_1(k+1) &= -a_1x_1(k) - a_2x_2(k) - a_3x_3(k) + e(k), \\ x_2(k+1) &= x_1(k), \\ x_3(k+1) &= x_2(k). \end{aligned} \quad (4.22)$$

Using vector-matrix notation,⁷ we can write this in the compact form

$$\mathbf{x}(k+1) = \mathbf{A}_c \mathbf{x}(k) + \mathbf{B}_c e(k),$$

where

$$\begin{aligned} \mathbf{x} &= \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \\ \mathbf{A}_c &= \begin{bmatrix} -a_1 & -a_2 & -a_3 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \end{aligned} \quad (4.23)$$

and

$$\mathbf{B}_c = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}. \quad (4.24)$$

The output equation is also immediate except that we must watch to catch *all* paths by which the state variables combine in the output. The problem is caused

⁷ We assume the reader has some knowledge of matrices. The results we require and references to study material are given in Appendix C.

Figure 4.8

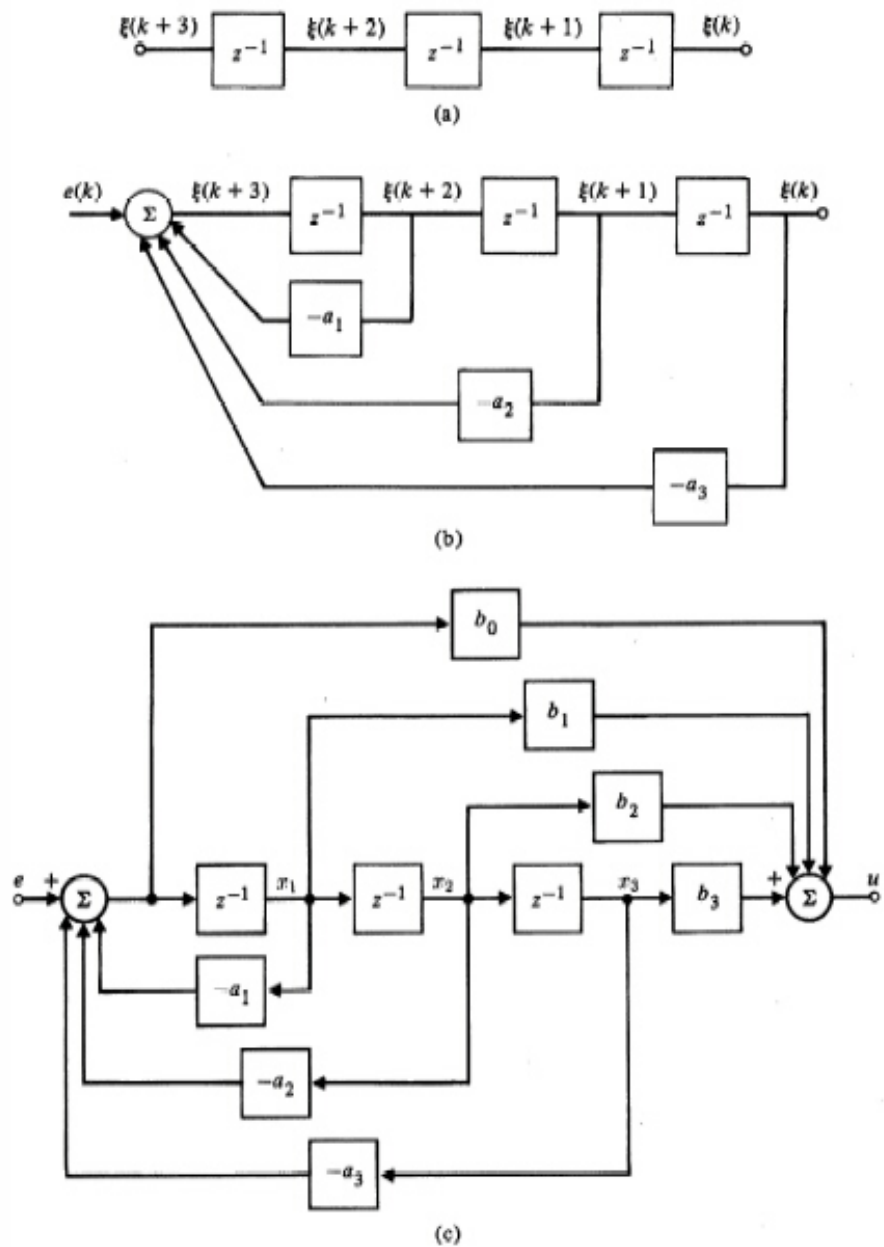
Block diagram development of control canonical form:

(a) Solving for $\xi(k)$;

(b) solving for $\xi(k+3)$

from $e(k)$ and past ξ 's;

(c) solving for $U(k)$ from ξ 's



by the b_0 term. If $b_0 = 0$, then $u = b_1x_1 + b_2x_2 + b_3x_3$, and the corresponding matrix form is immediate. However, if b_0 is not 0, x_1 for example not only reaches the output through b_1 but also by the parallel path with gain $-b_0a_1$. The complete equation is

$$u = (b_1 - a_1b_0)x_1 + (b_2 - a_2b_0)x_2 + (b_3 - a_3b_0)x_3 + b_0e.$$

In vector/matrix notation, we have

$$u = \mathbf{C}_c \mathbf{x} + \mathbf{D}_c e$$

where

$$\mathbf{C}_c = [b_1 - a_1b_0 \quad b_2 - a_2b_0 \quad b_3 - a_3b_0] \quad (4.25)$$

$$\mathbf{D}_c = b_0. \quad (4.26)$$

We can combine the equations for the state evolution and the output to give the very useful and most compact equations for the dynamic system,

$$\mathbf{x}(k+1) = \mathbf{A}_c \mathbf{x}(k) + \mathbf{B}_c e(k),$$

where $\mathbf{A}_c$ and $\mathbf{B}_c$ for this control canonical form are given by Eq. (4.23), and $\mathbf{C}_c$ and $\mathbf{D}_c$ are given by Eq. (4.25).

observer canonical form The other canonical form we want to illustrate is called the “**observer**” **canonical form** and is found by starting with the difference equations in operator/transform form as

$$z^3u + a_1z^2u + a_2zu + a_3u = b_0z^3e + b_1z^2e + b_2ze + b_3e.$$

In this equation, the external input is $e(k)$, and the response is $u(k)$, which is the solution of this equation. The terms with factors of z are time-shifted toward the future with respect to k and must be eliminated in some way. To do this, we assume at the start that we have the $u(k)$, and of course the $e(k)$, and we rewrite the equation as

$$b_3e - a_3u = z^3u + a_1z^2u + a_2zu - b_0z^3e - b_1z^2e - b_2ze.$$

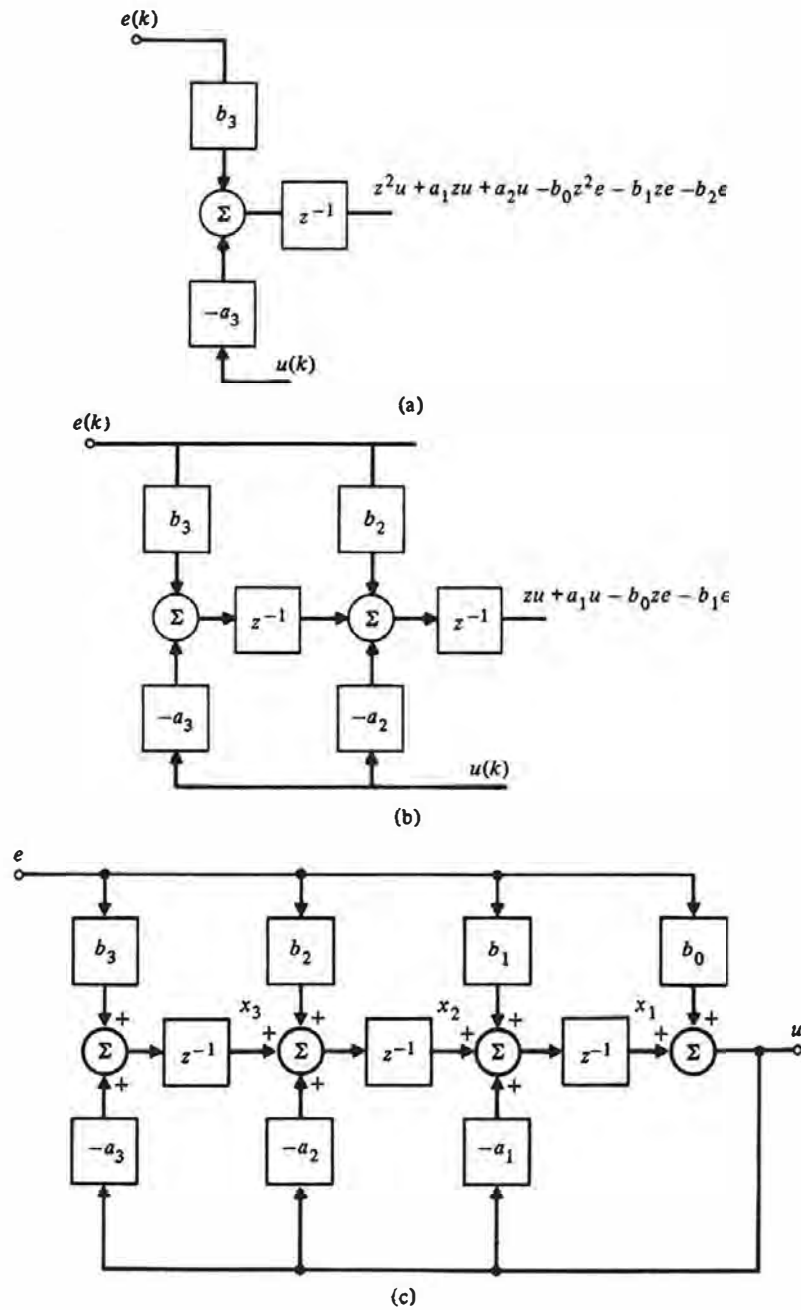
Here, *every* term on the right is multiplied by at least one power of z , and thus we can operate on the lot by z^{-1} as shown in the partial block diagram drawn in Fig. 4.9(a).

Now in this internal result there appear a_2u and $-b_2e$, which can be canceled by adding proper multiples of u and e , as shown in Fig. 4.9(b), and once they have been removed, the remainder can again be operated on by z^{-1} .

If we continue this process of subtracting out the terms at k and operating on the rest by z^{-1} , we finally arrive at the place where all that is left is u alone! But that is just what we assumed we had in the first place, so connecting this term back to the start finishes the block diagram, which is drawn in Fig. 4.9(c).

Figure 4.9

Block diagram development of observer canonical form: (a) the first partial sum and delay; (b) the second partial sum and delay; (c) the completion with the solution for $u(k)$



A preferred choice of numbering for the state components is also shown in the figure. Following the technique used for the control form, we find that the matrix equations are given by

$$\begin{aligned} \mathbf{x}(k+1) &= \mathbf{A}_o \mathbf{x}(k) + \mathbf{B}_o e(k) \\ u(k) &= \mathbf{C}_o \mathbf{x}(k) + \mathbf{D}_o e(k). \end{aligned} \quad (4.27)$$

where

$$\begin{aligned} \mathbf{A}_o &= \begin{bmatrix} -a_1 & 1 & 0 \\ -a_2 & 0 & 1 \\ -a_3 & 0 & 0 \end{bmatrix} \\ \mathbf{B}_o &= \begin{bmatrix} b_1 - b_0 a_1 \\ b_2 - b_0 a_2 \\ b_3 - b_0 a_3 \end{bmatrix} \\ \mathbf{C}_o &= [1 \ 0 \ 0] \\ \mathbf{D}_o &= [b_0]. \end{aligned}$$

The block diagrams of Figs. 4.8 and 4.9 are called **direct canonical form** realizations of the transfer function $H(z)$ because the gains of the realizations are coefficients in the transfer-function polynomials.

cascade canonical form

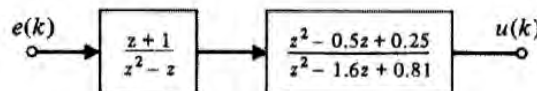
Another useful form is obtained if we realize a transfer function by placing several first- or second-order direct forms in series with each other, a **cascade canonical form**. In this case, the $H(z)$ is represented as a product of factors, and the poles and zeros of the transfer function are clearly represented in the coefficients.

For example, suppose we have a transfer function

$$\begin{aligned} H(z) &= \frac{z^3 + 0.5z^2 - 0.25z + 0.25}{z^4 - 2.6z^3 + 2.4z^2 - 0.8z} \\ &= \frac{(z+1)(z^2 - 0.5z + 0.25)}{(z^2 - z)(z^2 - 1.6z + 0.8)}. \end{aligned}$$

The zero factor $z+1$ can be associated with the pole factor $z^2 - z$ to form one second-order system, and the zero factor $z^2 - 0.5z + 0.25$ can be associated with the second-order pole factor $z^2 - 1.6z + 0.8$ to form another. The cascade factors, which could be realized in a direct form such as control or observer form, make a cascade form as shown in Fig. 4.10.

Figure 4.10
Block diagram of a
cascade realization



4.2.4 Relation of Transfer Function to Pulse Response

We have shown that a transfer function of z^{-1} is a unit delay in the time domain. We can also give a time-domain meaning to an arbitrary transfer function. Recall that the z -transform is defined by Eq. (4.8) to be $E(z) = \sum e_k z^{-k}$, and the transfer function is defined from Eq. (4.16) as $H(z)$ when the input and output are related by $U(z) = H(z)E(z)$. Now suppose we deliberately select $e(k)$ to be the unit discrete pulse defined by

$$\begin{aligned} e_k &= \begin{cases} 1, & (k = 0), \\ 0, & (k \neq 0), \end{cases} \\ &\triangleq \delta_k. \end{aligned} \quad (4.28)$$

Then it follows that $E(z) = 1$ and therefore that

$$U(z) = H(z). \quad (4.29)$$

Thus the transfer function $H(z)$ is seen to be the *transform* of the response to a unit-pulse input. For example, let us look at the system of Fig. 4.4 and put a unit pulse in at the e_k -node (with no signals in the system beforehand).⁸ We can readily follow the pulse through the block and build Table 4.1.

Thus the unit-pulse response is zero for negative k , is $T/2$ at $k = 0$, and equals T thereafter. The z -transform of this sequence is

$$H(z) = \sum_{-\infty}^{\infty} u_k z^{-k} \triangleq \sum_{-\infty}^{\infty} h_k z^{-k}.$$

Table 4.1

Step-by-step construction of the unit pulse response for Fig. 4.4

k	e_{k-1}	e_k	a_k	u_{k-1}	$u_k \equiv h_k$
0	0	1	$T/2$	0	$T/2$
1	1	0	$T/2$	$T/2$	T
2	0	0	0	T	T
3	0	0	0	T	T

⁸ In this development we assume that Eq. (4.7) is intended to be used as a formula for computing values of u_k as k increases. There is no reason why we could not also solve for u_k as k takes on negative values. The direction of time comes from the application and not from the recurrence equation.

If we add $T/2$ to the z^0 -term and subtract $T/2$ from the whole series, we have a simpler sum, as follows

$$\begin{aligned}
 H(z) &= \sum_{k=0}^{\infty} Tz^{-k} - \frac{T}{2} \\
 &= \frac{T}{1 - z^{-1}} - \frac{T}{2} \quad (1 < |z|) \\
 &= \frac{2T - T(1 - z^{-1})}{2(1 - z^{-1})} \\
 &= \frac{T + Tz^{-1}}{2(1 - z^{-1})} \\
 &= \frac{T}{2} \frac{z + 1}{z - 1} \quad (1 < |z|). \tag{4.30}
 \end{aligned}$$

Of course, this is the transfer function we obtained in Eq. (4.13) from direct analysis of the difference equation.

A final point of view useful in the interpretation of the discrete transfer function is obtained by multiplying the infinite polynomials of $E(z)$ and $H(z)$ as suggested in Eq. (4.16). For purposes of illustration, we will assume that the unit-pulse response, h_k , is zero for $k < 0$. Likewise, we will take $k = 0$ to be the starting time for e_k . Then the product that produces $U(z)$ is the polynomial product given in Fig. 4.11.

Figure 4.11

Representation of the product $E(z)H(z)$ as a product of polynomials

e_0	$+e_1z^{-1}$	$+e_2z^{-2}$	$+e_3z^{-3}$	$+\dots$
h_0	$+h_1z^{-1}$	$+h_2z^{-2}$	$+h_3z^{-3}$	$+\dots$
e_0h_0	$+e_1h_0z^{-1}$	$+e_2h_0z^{-2}$	$+e_3h_0z^{-3}$	
	$+e_0h_1z^{-1}$	$+e_1h_1z^{-2}$	$+e_2h_1z^{-3}$	
		$+e_0h_2z^{-2}$	$+e_1h_2z^{-3}$	
			$+e_0h_3z^{-3}$	
$e_0h_0 + (e_0h_1 + e_1h_0)z^{-1} + (e_0h_2 + e_1h_1 + e_2h_0)z^{-2} + (e_0h_3 + e_1h_2 + e_2h_1 + e_3h_0)z^{-3} + \dots$				

Since this product has been shown to be $U(z) = \sum u_k z^{-k}$, it must therefore follow that the coefficient of z^{-k} in the product is u_k . Listing these coefficients, we have the relations

$$\begin{aligned} u_0 &= e_0 h_0 \\ u_1 &= e_0 h_1 + e_1 h_0 \\ u_2 &= e_0 h_2 + e_1 h_1 + e_2 h_0 \\ u_3 &= e_0 h_3 + e_1 h_2 + e_2 h_1 + e_3 h_0. \end{aligned}$$

The extrapolation of this simple pattern gives the result

$$u_k = \sum_{j=0}^k e_j h_{k-j}.$$

By extension, we let the lower limit of the sum be $-\infty$ and the upper limit be $+\infty$:

$$u_k = \sum_{j=-\infty}^{\infty} e_j h_{k-j}. \quad (4.31)$$

Negative values of j in the sum correspond to inputs applied before time equals zero. Values for j greater than k occur if the unit-pulse response is nonzero for negative arguments. By definition, such a system, which responds *before* the input that causes it occurs, is called **noncausal**. This is the discrete **convolution** sum and is the analog of the convolution integral that relates input and impulse response to output in linear, constant, continuous systems.

convolution

To verify Eq. (4.31) we can take the z -transform of both sides

$$\sum_{k=-\infty}^{\infty} u_k z^{-k} = \sum_{k=-\infty}^{\infty} z^{-k} \sum_{j=-\infty}^{\infty} e_j h_{k-j}.$$

Interchanging the sum on j with the sum on k leads to

$$U(z) = \sum_{j=-\infty}^{\infty} e_j \sum_{k=-\infty}^{\infty} z^{-k} h_{k-j}.$$

Now let $k - j = l$ in the second sum

$$U(z) = \sum_{j=-\infty}^{\infty} e_j \sum_{l=-\infty}^{\infty} h_l z^{-(l+j)},$$

but $z^{-(l+j)} = z^{-l} z^{-j}$, which leads to

$$U(z) = \sum_{j=-\infty}^{\infty} e_j z^{-j} \sum_{l=-\infty}^{\infty} h_l z^{-l},$$

and we recognize these two separate sums as

$$U(z) = E(z)H(z).$$

We can also derive the convolution sum from the properties of linearity and stationarity. First we need more formal definitions of “linear” and “stationary.”

1. **Linearity:** A system with input e and output u is *linear* if superposition applies, which is to say, if $u_1(k)$ is the response to $e_1(k)$ and $u_2(k)$ is the response to $e_2(k)$, then the system is linear if and only if, for every scalar α and β , the response to $\alpha e_1 + \beta e_2$ is $\alpha u_1 + \beta u_2$.
2. **Stationarity:** A system is *stationary*, or time invariant, if a time shift in the input results in only a time shift in the output. For example, if we take the system at rest (no internal energy in the system) and apply a certain signal $e(k)$, suppose we observe a response $u(k)$. If we repeat this experiment at any later time when the system is again at rest and we apply the shifted input, $e(k - N)$, if we see $u(k - N)$, then the system is stationary. A constant coefficient difference equation is stationary and typically referred to as a constant system.

These properties can be used to derive the convolution in Eq. (4.31) as follows. If response to a unit pulse at $k = 0$ is $h(k)$, then response to a pulse of intensity e_0 is $e_0 h(k)$ if the system is linear. Furthermore, if the system is stationary then a delay of the input will delay the response. Thus, if

$$\begin{aligned} e &= e_l, & k &= l \\ &= 0, & k &\neq l. \end{aligned}$$

Finally, by linearity again, the total response at time k to a sequence of these pulses is the *sum* of the responses, namely,

$$u_k = e_0 h_k + e_1 h_{k-1} + \cdots + e_l h_{k-l} + \cdots + e_k h_0,$$

or

$$u_k = \sum_{l=0}^k e_l h_{k-l}.$$

Now note that if the input sequence began in the distant past, we must include terms for $l < 0$, perhaps back to $l = -\infty$. Similarly, if the system should be noncausal, future values of e where $l > k$ may also come in. The general case is thus (again)

$$u_k = \sum_{l=-\infty}^{\infty} e_l h_{k-l}. \quad (4.32)$$

4.2.5 External Stability

A very important qualitative property of a dynamic system is stability, and we can consider internal or external stability. Internal stability is concerned with the responses at all the internal variables such as those that appear at the delay

elements in a canonical block diagram as in Fig. 4.8 or Fig. 4.9 (the state). Otherwise we can be satisfied to consider only the **external stability** as given by the study of the input–output relation described for the linear stationary case by the convolution Eq. (4.32). These differ in that some internal modes might not be connected to both the input and the output of a given system.

For external stability, the most common definition of *appropriate response* is that for every Bounded Input, we should have a Bounded Output. If this is true we say the system is BIBO stable. A test for BIBO stability can be given directly in terms of the unit-pulse response, h_k . First we consider a sufficient condition. Suppose the input e_k is bounded, that is, there is an M such that

$$|e_l| \leq M < \infty \quad \text{for all } l. \quad (4.33)$$

If we consider the magnitude of the response given by Eq. (4.32), it is easy to see that

$$|u_k| \leq \left| \sum e_l h_{k-l} \right|,$$

which is surely less than the sum of the magnitudes as given by

$$\leq \sum_{-\infty}^{\infty} |e_l| |h_{k-l}|.$$

But, because we assume Eq. (4.33), this result is in turn bounded by

$$\leq M \sum_{-\infty}^{\infty} |h_{k-l}|. \quad (4.34)$$

Thus the output will be bounded for every bounded input if

$$\sum_{l=-\infty}^{\infty} |h_{k-l}| < \infty. \quad (4.35)$$

This condition is also necessary, for if we consider the bounded (by 1!) input

$$\begin{aligned} e_l &= \frac{h_{-l}}{|h_{-l}|} & (h_{-l} \neq 0) \\ &= 0 & (h_{-l} = 0) \end{aligned}$$

and apply it to Eq. (4.32), the output at $k = 0$ is

$$\begin{aligned} u_0 &= \sum_{l=-\infty}^{\infty} e_l h_{-l} \\ &= \sum_{l=-\infty}^{\infty} \frac{(h_{-l})^2}{|h_{-l}|} \\ &= \sum_{l=-\infty}^{\infty} |h_{-l}|. \end{aligned} \quad (4.36)$$

Thus, unless the condition given by Eq. (4.35) is true, the system is not BIBO stable.

◆ Example 4.3 Integration Stability

Is the discrete approximation to integration (Eq. 4.7) BIBO stable?

Solution. The test given by Eq. (4.35) can be applied to the unit pulse response used to compute the u_k -column in Table 4.1. The result is

$$\begin{aligned} h_0 &= T/2 \\ h_k &= T, \quad k > 0 \\ \sum |h_k| &= T/2 + \sum_{k=1}^{\infty} T = \text{unbounded.} \end{aligned} \quad (4.37)$$

Therefore, this discrete approximation to integration is not BIBO stable!

◆ Example 4.4 General Difference Equation Stability

Consider the difference equation (4.2) with all coefficients except a_1 and b_0 equal to zero

$$u_k = a_1 u_{k-1} + b_0 e_k. \quad (4.38)$$

Is this equation stable?

Solution. The unit-pulse response is easily developed from the first few terms to be

$$\begin{aligned} u_0 &= b_0, & u_1 &= a_1 b_0, & u_2 &= a_1^2 b_0, & \dots \\ u_k &= h_k = b_0 a_1^k, & k &\geq 0. \end{aligned} \quad (4.39)$$

Applying the test, we have

$$\begin{aligned} \sum_{-\infty}^{\infty} |h_l| &= \sum_{l=0}^{\infty} b_0 |a_1|^l = b_0 \frac{1}{1 - |a|} \quad (|a| < 1) \\ &= \text{unbounded} \quad (|a| \geq 1). \end{aligned}$$

Thus we conclude that the system described by this equation is BIBO stable if $|a| < 1$, and unstable otherwise.

For a more general rational transfer function with many simple poles, we can expand the function in partial fractions about its poles, and the corresponding pulse response will be a sum of respective terms. As we saw earlier, if a pole

is inside the unit circle, the corresponding pulse response decays with time geometrically and is stable. Thus, if all poles are inside the unit circle, the system with rational transfer function is stable; if at least one pole is on or outside the unit circle, the corresponding system is not BIBO stable. With modern computer programs available, finding the poles of a particular transfer function is no big deal. Sometimes, however, we wish to test for stability of an entire class of systems; or, as in an adaptive control system, the potential poles are constantly changing and we wish to have a quick test for stability in terms of the literal polynomial coefficients. In the continuous case, such a test was provided by Routh; in the discrete case, the most convenient such test was worked out by Jury and Blanchard(1961).⁹

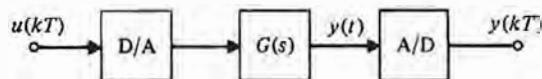
4.3 Discrete Models of Sampled-Data Systems

The systems and signals we have studied thus far have been defined in discrete time only. Most of the dynamic systems to be controlled, however, are continuous systems and, if linear, are described by continuous transfer functions in the Laplace variable s . The interface between the continuous and discrete domains are the A/D and the D/A converters as shown in Fig. 1.1. In this section we develop the analysis needed to compute the discrete transfer function between the samples that come from the digital computer to the D/A converter and the samples that are picked up by the A/D converter.¹⁰ The situation is drawn in Fig. 4.12.

4.3.1 Using the z -Transform

We wish to find the discrete transfer function from the input samples $u(kT)$ (which probably come from a computer of some kind) to the output samples $y(kT)$ picked up by the A/D converter. Although it is possibly confusing at first, we follow convention and call the discrete transfer function $G(z)$ when the continuous transfer function is $G(s)$. Although $G(z)$ and $G(s)$ are entirely different functions, they do describe the *same* plant, and the use of s for the continuous transform and z for the discrete transform is always maintained. To

Figure 4.12
The prototype
sampled-data system



⁹ See Franklin, Powell, and Workman, 2nd edition, 1990, for a discussion of the Jury test.

¹⁰ In Chapter 5, a comprehensive frequency analysis of sampled data systems is presented. Here we undertake only the special problem of finding the sample-to-sample discrete transfer function of a continuous system between a D/A and an A/D.